AYE

• PLC PROGRAMMING • LESSON 06

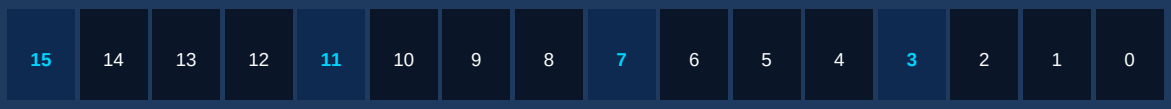
Data Handling & Move Instructions

MOV | MOVB | FILL | COP | Indirect Addressing

What You Will Learn

- How PLCs organise data in memory — bits, bytes, words, DWORDs
- MOV and Block Move instructions with real Siemens & AB examples
- FILL and COP instructions for bulk data operations
- Indirect addressing — using a pointer to read/write any address

PLC Data Memory Layout — Word Structure



Bit 15 (MSB) Bit 0 (LSB)

1 Word = 16 bits = 2 bytes | 1 DWord = 32 bits = 2 Words

Awet G. Nway

Founder, AYE Tech Hub | PLC & Industrial Automation Engineer

PLC-006: Data Handling & Move Instructions

Every PLC program works with data — sensor readings, calculated values, setpoints, counters, temperatures, speeds. Understanding how a PLC stores data in memory and how to move that data between locations is one of the most important skills in PLC programming. Lesson PLC-006 covers the complete set of data handling instructions used in modern PLCs.

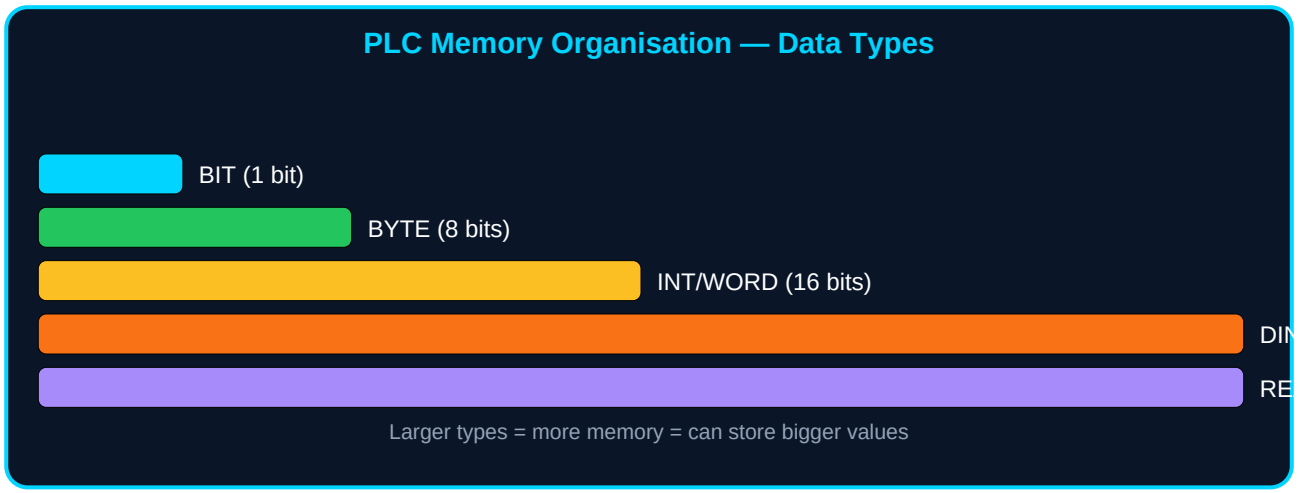
Learning Objectives

- Understand PLC memory organisation: bits, bytes, integers, double words, and reals
- Use the MOV instruction to copy single values between memory locations
- Use MOVB and MOVW for byte and word-level transfers
- Apply the FILL instruction to initialise an entire array in a single scan
- Use the COP (Copy File) instruction to transfer data blocks
- Understand and apply indirect addressing using a pointer variable

1. PLC Memory Organisation

Before learning move instructions, you must understand how a PLC organises data in its memory. All PLCs divide their memory into named areas, and each area holds a specific type of data. The size of a memory location determines the range of values it can hold.

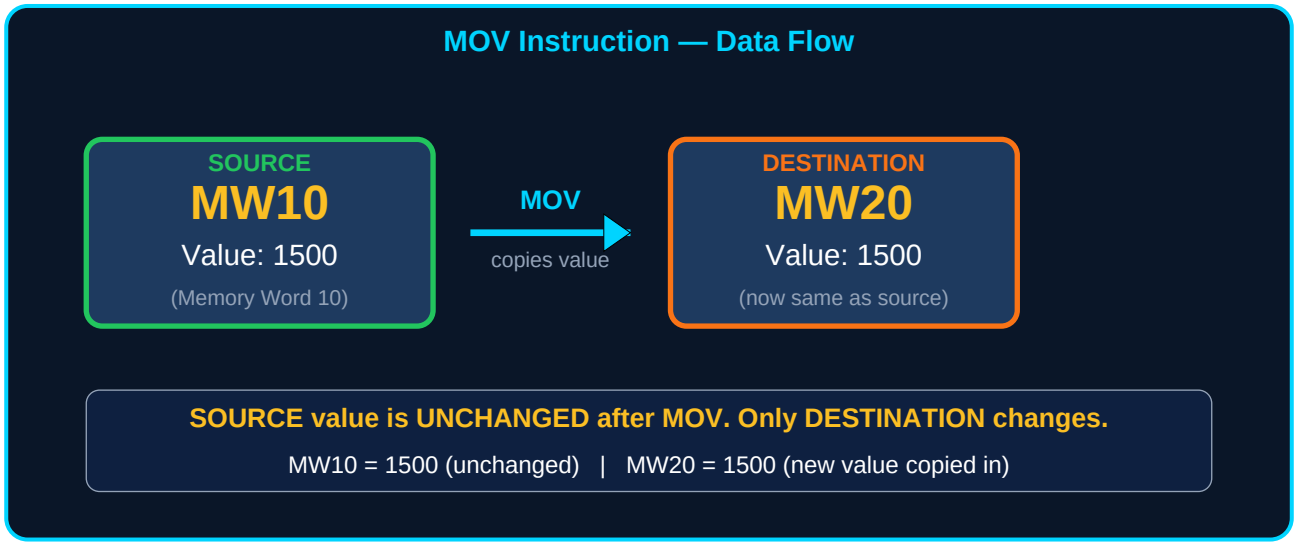
Data Type	Size	Range	Siemens TIA	Allen-Bradley	Use Case
BIT / BOOL	1 bit	0 or 1	M0.0	B3/0	Input/output flags, status bits
BYTE	8 bits	0 to 255	MB10	N/A direct	Small integers, BCD digits
INT / WORD	16 bits	–32768 to 32767	MW10 / DB1.DBW0	N7:0 (INT)	Counters, temperatures, setpoints
DINT / DWORD	32 bits	–2.1B to 2.1B	MD10 / DB1.DBD0	L9:0 (DINT)	High-precision values, large counters
REAL	32 bits	±3.4 × 10³■	MD10 (REAL)	F8:0 (REAL)	Decimal values: 3.14, 98.6°C, 4.20mA



The key principle: a larger data type uses more memory but stores a larger range of values. Choose the smallest type that fits your data — an overload trip counter rarely exceeds 1000, so INT is more than sufficient. A real-world flow measurement in cubic metres per hour might need a REAL (floating point).

2. The MOV (Move) Instruction

The MOV instruction is the most fundamental data handling instruction. It copies a value from a source location to a destination location. The source value is **not changed** — MOV *copies*, it does not move in the "remove from source" sense of the word.



Element	Siemens TIA Portal	Allen-Bradley Studio 5000	Notes
Instruction name	`MOVE`	`MOV`	Same function, different label
Enable input (EN)	Rung condition must be TRUE to execute	Same	Use a contact or always-ON coil as enable
Source (IN)	`MW10 (word) MD10 (double word) #constant (e.g. 1500)`	`N7:0 (integer) Source value or address`	Can be a constant or a memory address
Destination (OUT)	`MW20 (word) DB1.DBW4`	`N7:5 (destination integer)`	Must match data type of source
Result	OUT = copy of IN. IN unchanged.	Same	No arithmetic. Pure copy.

MOV Practical Examples

- **Copying a setpoint:** MOV constant 1500 → MW_SpeedSetpoint. Every scan, MW_SpeedSetpoint will hold 1500 (the desired motor speed in rpm) until a different MOV instruction writes a new value.

- **Saving a reading:** When a batch starts (rising edge on M_BatchStart.0), execute MOV MD_CurrentTemp → MD_BatchStartTemp. This saves the temperature at batch start for comparison throughout the batch.
- **Error code logging:** When a fault occurs, MOV the fault code integer into a data block register for the HMI to display. MOV #FAULT_CODE → DB_Faults.DBW_LastFaultCode.
- **Initialising outputs:** At program start (first scan flag), MOV 0 to every output word to ensure all outputs start in a known, safe state.

The source and destination in a MOV instruction should be compatible data types. Attempting to MOV a 32-bit value into a 16-bit word will cause a truncation error or program fault on most PLCs. If you need to convert data types, use conversion instructions (INT_TO_REAL, REAL_TO_INT) *before* the MOV.

3. FILL — Block Fill Instruction

The FILL instruction writes the same value to a consecutive block of memory words in a single instruction. Instead of writing 20 individual MOV instructions to clear 20 registers, one FILL instruction does it in one PLC scan cycle. FILL is used to initialise arrays, reset data blocks, and clear history buffers.

Parameter	Siemens TIA Portal	Allen-Bradley (FLL)	Description
Source (IN)	`IN: value to fill (e.g. 0, 100, #MY_CONST)`	`Source: value or address`	The single value that will be written to every destination word
Count (N)	`N: number of elements to fill (e.g. 20)`	`Length: N words/bytes`	How many consecutive words to write the value to
Destination (OUT)	`OUT: first destination address (e.g. MW100)`	`Dest: first address`	FILL writes to OUT, OUT+1, OUT+2 ... OUT+N-1
Data type	Must match — FILL INT fills INT words, FILL REAL fills REAL DWords	Same	Never mix data types across the destination range

- **Array initialisation:** On system startup, FILL 0 into all data registers to guarantee no garbage values from previous runs cause incorrect behaviour.
- **Recipe loading:** FILL a default recipe value into a parameter array when a new product type is selected, then overwrite only the parameters that differ.
- **History buffer reset:** Before starting a new batch trend, FILL 0 into the 100-word trend buffer so previous data does not appear in the new trend.

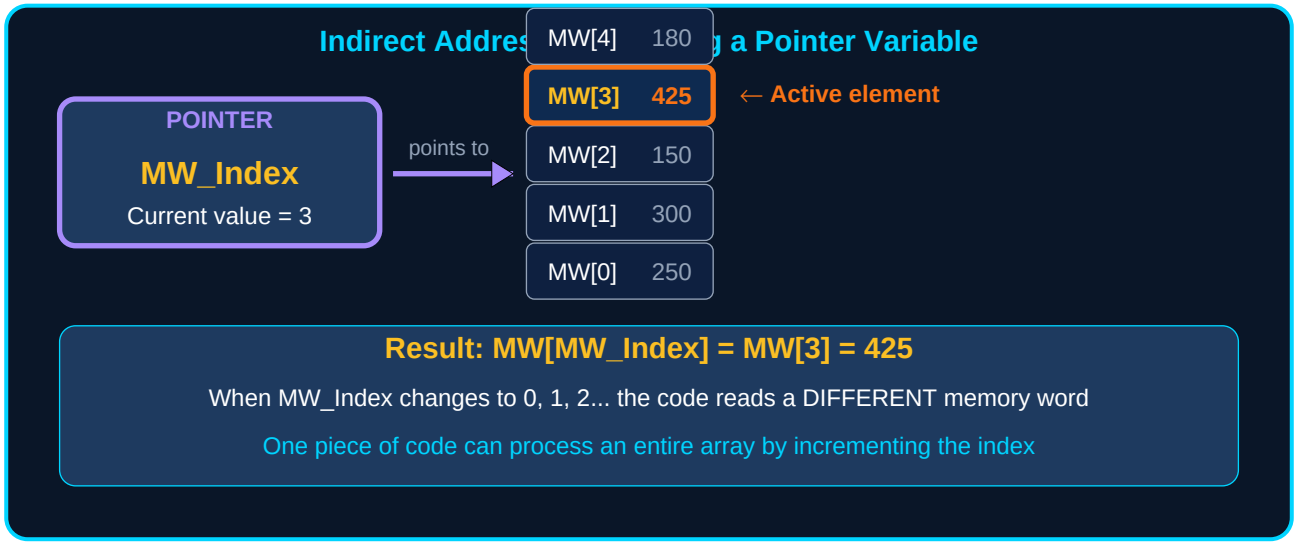
4. COP — Copy File Instruction

COP (Copy File / Block Copy) copies a block of consecutive memory locations from a source start address to a destination start address. Unlike FILL (which writes one value to many locations), COP copies many different values to a new location in memory — it is the PLC equivalent of a memcpy() function in C programming.

Parameter	Siemens (BLKMOV)	Allen-Bradley (COP)	Description
Source (SRC)	`SRC: first source address`	`Source: file or address`	Start of the block to be copied FROM
Count (N)	`COUNT: number of elements`	`Length: N elements`	How many consecutive words to copy
Destination (DEST)	`DEST: first destination`	`Dest: destination file`	Where the copied block will be written TO
Common use	Recipe changeover — copy active recipe from recipe DB to runtime DB	Copying data tables, recipe files	Source and destination may not overlap

5. Indirect Addressing — Using a Pointer

Direct addressing means specifying a fixed memory location: "read MW10." Indirect addressing means specifying an address that is calculated at runtime: "read MW[Index]" where Index is itself a variable stored in memory. Indirect addressing is the key technique for processing arrays of data with a single block of PLC code.



PLC Platform	Method	Syntax Example	How It Works
Siemens TIA Portal	Array index in DB	`DB1.DataArray[MW_Index]`	MW_Index is an INT pointer. Change MW_Index value to access different array elements.
Siemens TIA Portal	Pointer (P#) addressing	`L P#M10.0 SLD 3 ; multiply by 8 T MD_Pointer`	Create a memory pointer, offset it, use MOVE with pointer for direct byte addressing
Allen-Bradley	Indirect file addressing	`N7:[N7:0] or F8:[N7:5]`	N7:0 contains the element number. N7:[N7:0] reads the element AT that index.
Allen-Bradley	Indexed addressing	`N10[N7:0]:0`	N7:0 is the index into file N10. Increment N7:0 to step through N10.

Practical: Reading a Recipe Array

Consider a packaging machine with 10 different product recipes. Each recipe has 5 parameters (speed, fill weight, seal temperature, label offset, conveyor delay). You could write 50 MOV instructions to handle all 10 recipes manually — or you could use indirect addressing.

Step	What Happens	Code Concept
1. Operator selects recipe	HMI writes recipe number (0–9) to MW_RecipeNum	`MW_RecipeNum := 3 (recipe 3)`
2. Calculate base address	Multiply recipe number × 5 (parameters per recipe) to get array offset	`MW_Index := MW_RecipeNum * 5`
3. Read parameters	Use indirect addressing to read 5 consecutive words starting at MW_Index offset	`BLKMOV DB_Recipes.Params[MW_Index], COUNT:=5, DEST:=DB_Runtime.ActiveRecipe[0]`
4. Load to outputs	Active recipe parameters are now in runtime DB, used by motion and dosing code	All output setpoints read from DB_Runtime.ActiveRecipe[]

6. Best Practices — Data Handling

- **Initialise all data on first scan:** Use the First Scan flag (SM0.1 in S7 / S:FS in AB) to FILL 0 to all output and control registers. Prevents "garbage" values from causing unexpected behaviour on startup.
- **Match data types strictly:** A type mismatch in a MOV instruction will cause a runtime fault on many PLCs. Always verify source and destination types match before going live.
- **Never leave array bounds unchecked:** With indirect addressing, always validate that your index is within the valid array range before executing the indirect access. An out-of-bounds access corrupts adjacent memory.
- **Document every data block structure:** In the TIA Portal DB properties or in your program documentation, record exactly what data each word offset represents. Memory at MW100 is meaningless without a comment saying "Active speed setpoint in rpm".
- **Use symbolic names, not raw addresses:** Name your data (SpeedSetpoint, BatchWeight, RecipeIndex) instead of using raw MW10, MW12. Symbolic names make programs readable and maintainable.

7. Practice Exercises

Complete these exercises in the AYE Tech Hub PLC Simulator or in a real PLC development environment (Siemens TIA Portal PLCSIM or Allen-Bradley Studio 5000 Logix Emulate).

#	Exercise	Instructions / Objective
1	MOV on rising edge	On the rising edge of I0.0, copy the value 250 into MW50. Verify in the watch table that MW50 = 250 only after the rising edge, not continuously.
2	Conditional MOV	If MW_Temperature > 80, MOV 1 (alarm code) into MW_AlarmCode. If MW_Temperature ≤ 80, MOV 0. Observe the output changing as you modify the simulated temperature value.
3	FILL array reset	On first scan, FILL 0 into MW100 through MW119 (20 words). Verify in the memory monitor that all 20 locations read 0 before any other code runs.
4	COP recipe changeover	Create two recipe arrays (Recipe0[5] and Recipe1[5]) with different integer values. When a "Recipe Select" input changes, COP the selected recipe into an "Active Recipe" array. Verify the active values match the selected recipe.
5	Indirect array scan	Store 8 integers in MW200–MW214 (every 2 bytes). Write a loop using an INT counter (MW_Loop = 0 to 7) and indirect addressing to find the maximum value in the array. Store the result in MW_MaxValue.

Lesson Summary

Data handling instructions are used in almost every professional PLC program. Recipe management, trend logging, alarm handling, and process control all depend on being able to move, copy, and process data efficiently across the PLC's memory.

Instruction	Function	When to Use
MOV	Copy one value from source to destination	Saving readings, loading setpoints, initialising variables
MOVB / MOVW	Move specific byte or word of a larger data unit	Working with BCD values, bit-packed data, protocol bytes
FILL / FLL	Write one value to N consecutive memory locations	Array initialisation, buffer reset, default recipe loading
COP / BLKMOV	Copy N values from source block to destination block	Recipe changeover, data logging, buffer shifting
Indirect addressing	Access memory location determined at runtime by a pointer variable	Array processing, recipe systems, data logging, loop-based operations

What Is Next — PLC-007

PLC-007: Math and Comparison Instructions — ADD, SUB, MUL, DIV, MOD instructions for arithmetic processing. CMP, GRT, LES, EQU, NEQ instructions for comparing values to make control decisions. Scaling analogue inputs (4–20mA → engineering units) using math instructions. Real-world applications in PID pre-processing, alarm triggering, and production counting.

Resource	Details
PDF Series	PLC-001 through PLC-030 — full curriculum available at ayetechub.com/pdfs.html
Practice Tool	AYE Tech Hub PLC Ladder Logic Simulator — free interactive tool at ayetechub.com/downloads
Community	Ask questions and share your PLC programs in the AYE Tech Hub Telegram group: t.me/ayetechub